Sistemas distribuidos

Laboratorio S11

Cortés Bolaños Angel David - 423066940

1. El punto de partida

Antes de implementar cualquier optimización, se comprobó el comportamiento predeterminado del sistema. Al solicitar el saldo de la cuenta de Ana 10 veces seguidas, se observó que las 10 consultas generaron lecturas directas a la base de datos relacional PostgreSQL. Esto demostró que, sin una caché, la base de datos principal soporta una carga innecesaria para datos que cambian con menor frecuencia de la que se consultan.

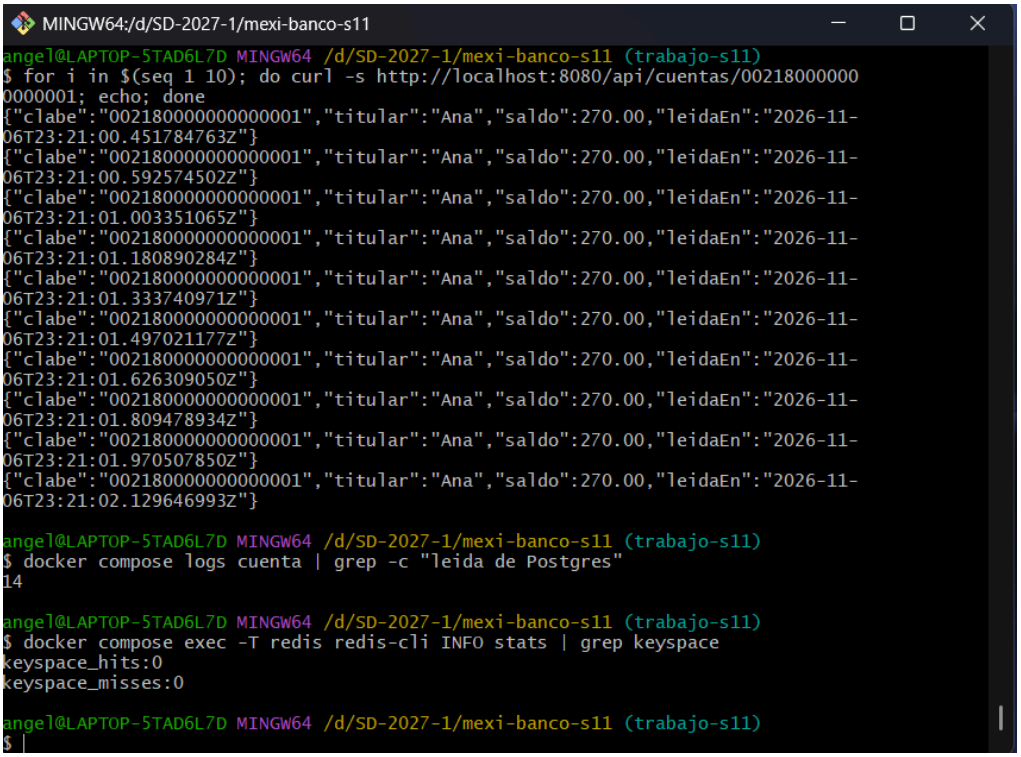


Figura 1. se hace una transferencia y se pide el saldo de Ana para demostrar que todas las consultas golpean directamente a la base de datos PostgreSQL.

2. La caché: Código y comportamiento

Se implementó el patrón cache-aside utilizando Spring y Redis. En el código, se configuró RedisCacheConfiguration para almacenar los datos en formato JSON (lo que permite su lectura directa) y un tiempo de vida (TTL) de 30 segundos. Al colocar la anotación @Cacheable en ConsultaDeSaldo, el sistema busca primero en Redis y, si falla (miss), lee de Postgres y guarda la copia. Debido a que Redis se ejecuta como un servicio externo, las tres instancias de la aplicación comparten la misma caché, evitando lecturas redundantes. Finalmente, mediante @CacheEvict en los métodos de cargo y abono, se garantiza que la copia se elimine (DEL) justo después del commit en Postgres, evitando mostrar datos desactualizados.

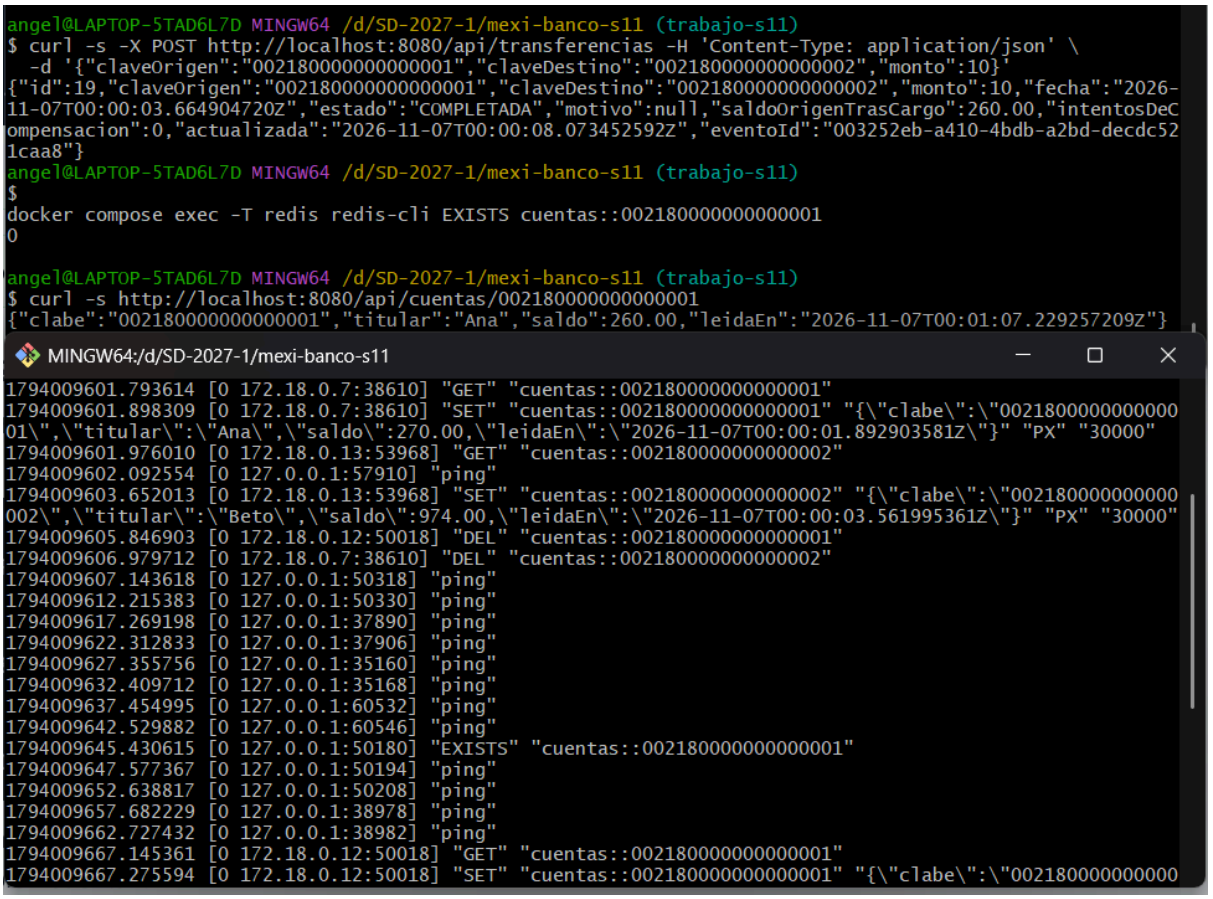


Figura 2. Borró a Ana y a Beto porque sus saldos cambiaron. Al final, se ve el último "GET" (que falló porque ya no estaba) seguido de un "SET" guardando el saldo nuevo con 30 segundos de vida

3. Hit ratio y latencia

Se realizaron pruebas de estrés para medir la efectividad de la caché. En el Escenario A (solo lecturas), se alcanzó un hit ratio del 99%, ya que Redis contestó casi todas las peticiones tras el primer guardado. En el Escenario B, al intercalar transferencias, el hit ratio bajó al ~80% debido a que cada movimiento invalida la caché y fuerza una nueva lectura a Postgres. Aunque los tiempos de latencia mostraron una mejora mínima (la base de datos local es rápida), la verdadera ganancia fue de capacidad: Redis absorbió cientos de peticiones HTTP, liberando a PostgreSQL para procesar exclusivamente operaciones de escritura.

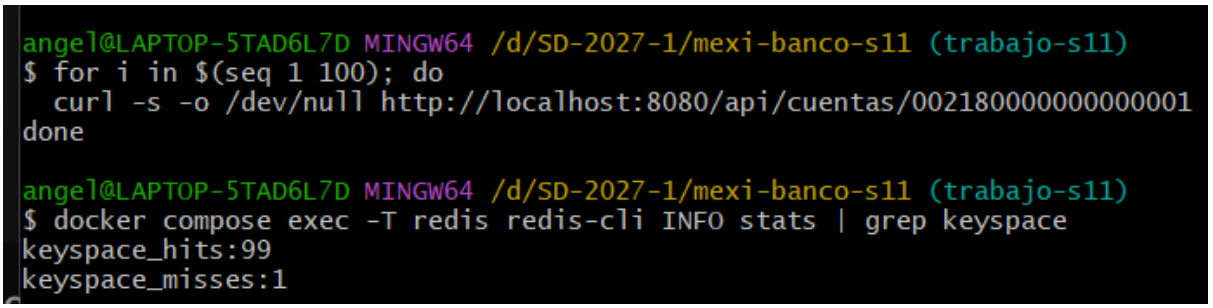


Figura 3. Valor de A

```
angel@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli CONFIG RESETSTAT
OK

angel@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco-s11 (trabajo-s11)
$ for j in $(seq 1 10); do
  for i in $(seq 1 10); do curl -s -o /dev/null http://localhost:8080/api/cuentas/002180000000000001; d
one
  curl -s -o /dev/null -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/j
son' -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1}'
done

angel@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli INFO stats | grep keyspace
keyspace_hits:100
keyspace_misses:20
```

Figura 4. Valor de B

```
angel@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 100); do   curl -s -o /dev/null -w '%{time_total}\n' http://localhost:8080/api/cuent
as/002180000000000001; done   | sort -n   | awk '{t[NR]=$1*1000; s+=$1*1000}
        END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0
.95)]}'
promedio=52.62 ms mediana=14.26 ms p95=29.66 ms

angel@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 100); do   curl -s -o /dev/null -w '%{time_total}\n' http://localhost:8080/api/cuent
as/002180000000000001; done   | sort -n   | awk '{t[NR]=$1*1000; s+=$1*1000}
        END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0
.95)]}'
promedio=14.63 ms mediana=13.15 ms p95=23.66 ms

angel@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 100); do   curl -s -o /dev/null -w '%{time_total}\n' http://localhost:8080/api/cuent
as/002180000000000001; done   | sort -n   | awk '{t[NR]=$1*1000; s+=$1*1000}
        END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0
.95)]}'
promedio=17.57 ms mediana=12.97 ms p95=23.09 ms

angel@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco-s11 (trabajo-s11)
$ CACHE_TIPO=none docker compose up -d cuenta
[+] up 6/6
 ✔ Container mexi-banco-v06-redis-1     Healthy                                          5.0s
 ✔ Container mexi-banco-v06-discovery-1 Healthy                                          5.0s
 ✔ Container mexi-banco-v06-db-1        Healthy                                          5.0s
 ✔ Container mexi-banco-v06-cuenta-1    Started                                          6.0s
 ✔ Container mexi-banco-v06-cuenta-2    Started                                          5.6s
 ✔ Container mexi-banco-v06-cuenta-3    Started                                          5.4s

angel@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 100); do   curl -s -o /dev/null -w '%{time_total}\n' http://localhost:8080/api/cuent
as/002180000000000001; done   | sort -n   | awk '{t[NR]=$1*1000; s+=$1*1000}
        END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0
.95)]}'
promedio=36.82 ms mediana=16.15 ms p95=29.41 ms

angel@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 100); do   curl -s -o /dev/null -w '%{time_total}\n' http://localhost:8080/api/cuent
as/002180000000000001; done   | sort -n   | awk '{t[NR]=$1*1000; s+=$1*1000}
        END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0
.95)]}'
promedio=16.46 ms mediana=14.47 ms p95=23.77 ms

angel@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 100); do   curl -s -o /dev/null -w '%{time_total}\n' http://localhost:8080/api/cuent
as/002180000000000001; done   | sort -n   | awk '{t[NR]=$1*1000; s+=$1*1000}
        END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0
.95)]}'
promedio=14.51 ms mediana=13.49 ms p95=19.45 ms
```

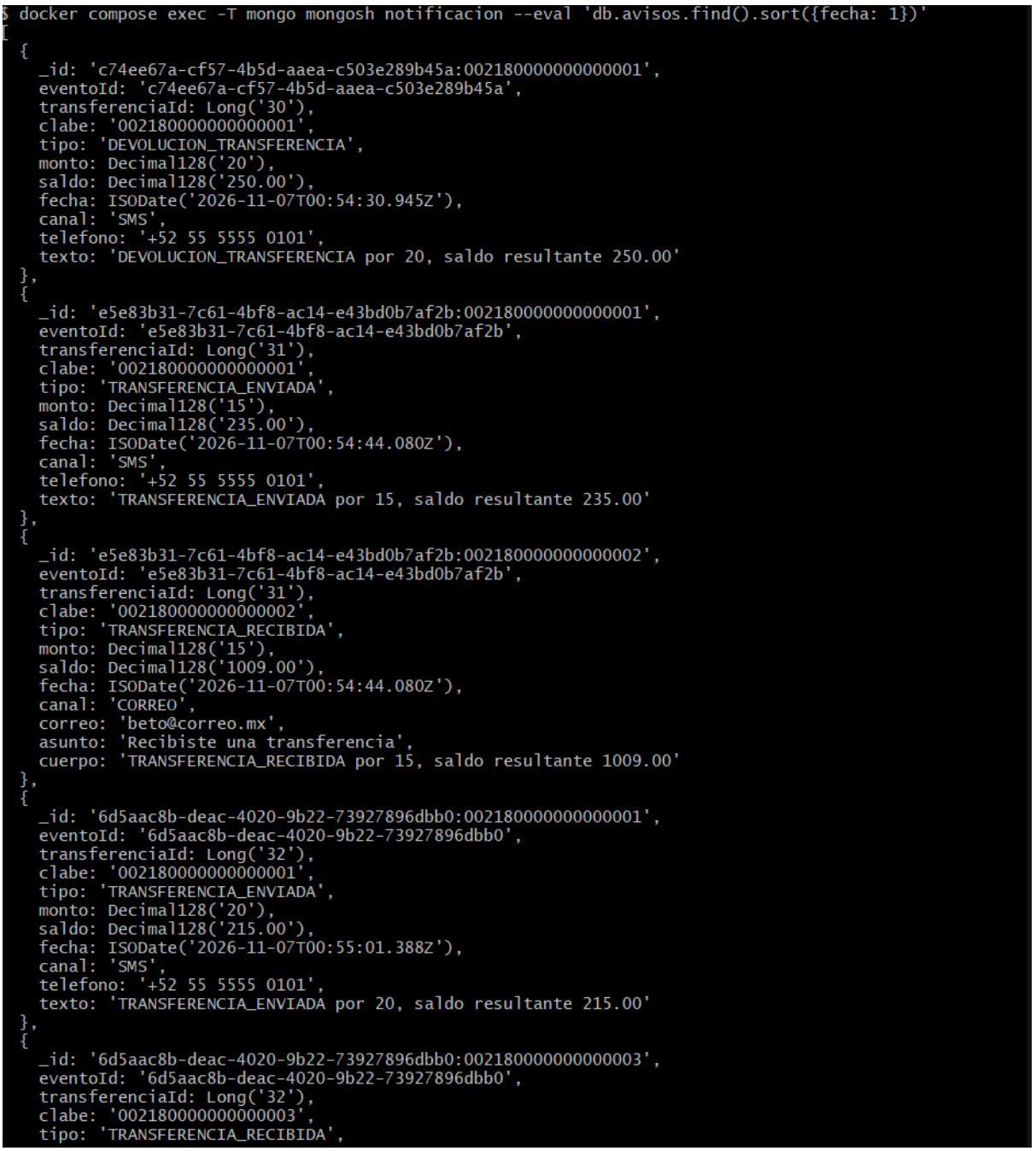
Figura 5. Promedios de latencia con/sin caché.

4. Los avisos en MongoDB

Para solucionar el problema de los esquemas rígidos en SQL, la bitácora de avisos se migró a MongoDB. Esto permitió almacenar documentos con estructuras variables en la misma colección: campos para SMS (teléfono), correo o notificaciones push (token) conviviendo sin necesidad de usar valores nulos. Además, se probó exitosamente el endpoint que limita la respuesta a los 10 avisos más recientes. El análisis de rendimiento mediante explain demostró la eficiencia del índice compuesto (clabe_fecha); con él (IXSCAN), la base de datos examinó exactamente 10 documentos en lugar de recorrer toda la colección (COLLSCAN).

```
angel@LAPTOP-5TAD6L7D MINGW64 /d/SD-2027-1/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({telefono: {$exists: true}},
 {clabe: 1, canal: 1, telefono: 1})'
[
  {
    _id: 'c74ee67a-cf57-4b5d-aaea-c503e289b45a:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: 'e5e83b31-7c61-4bf8-ac14-e43bd0b7af2b:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: '6d5aac8b-deac-4020-9b22-73927896dbb0:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  }
]
```

Figura 6. Respuesta de Mongo



```
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find().sort({fecha: 1})'
[
  {
    _id: 'c74ee67a-cf57-4b5d-aaea-c503e289b45a:002180000000000001',
    eventoId: 'c74ee67a-cf57-4b5d-aaea-c503e289b45a',
    transferenciaId: Long('30'),
    clabe: '002180000000000001',
    tipo: 'DEVOLUCION_TRANSFERENCIA',
    monto: Decimal128('20'),
    saldo: Decimal128('250.00'),
    fecha: ISODate('2026-11-07T00:54:30.945Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'DEVOLUCION_TRANSFERENCIA por 20, saldo resultante 250.00'
  },
  {
    _id: 'e5e83b31-7c61-4bf8-ac14-e43bd0b7af2b:002180000000000001',
    eventoId: 'e5e83b31-7c61-4bf8-ac14-e43bd0b7af2b',
    transferenciaId: Long('31'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('15'),
    saldo: Decimal128('235.00'),
    fecha: ISODate('2026-11-07T00:54:44.080Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 15, saldo resultante 235.00'
  },
  {
    _id: 'e5e83b31-7c61-4bf8-ac14-e43bd0b7af2b:002180000000000002',
    eventoId: 'e5e83b31-7c61-4bf8-ac14-e43bd0b7af2b',
    transferenciaId: Long('31'),
    clabe: '002180000000000002',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('15'),
    saldo: Decimal128('1009.00'),
    fecha: ISODate('2026-11-07T00:54:44.080Z'),
    canal: 'CORREO',
    correo: 'beto@correo.mx',
    asunto: 'Recibiste una transferencia',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 15, saldo resultante 1009.00'
  },
  {
    _id: '6d5aac8b-deac-4020-9b22-73927896dbb0:002180000000000001',
    eventoId: '6d5aac8b-deac-4020-9b22-73927896dbb0',
    transferenciaId: Long('32'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('20'),
    saldo: Decimal128('215.00'),
    fecha: ISODate('2026-11-07T00:55:01.388Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 20, saldo resultante 215.00'
  },
  {
    _id: '6d5aac8b-deac-4020-9b22-73927896dbb0:002180000000000003',
    eventoId: '6d5aac8b-deac-4020-9b22-73927896dbb0',
    transferenciaId: Long('32'),
    clabe: '002180000000000003',
    tipo: 'TRANSFERENCIA_RECIBIDA',
```

Figura 7. Respuesta de Mongo

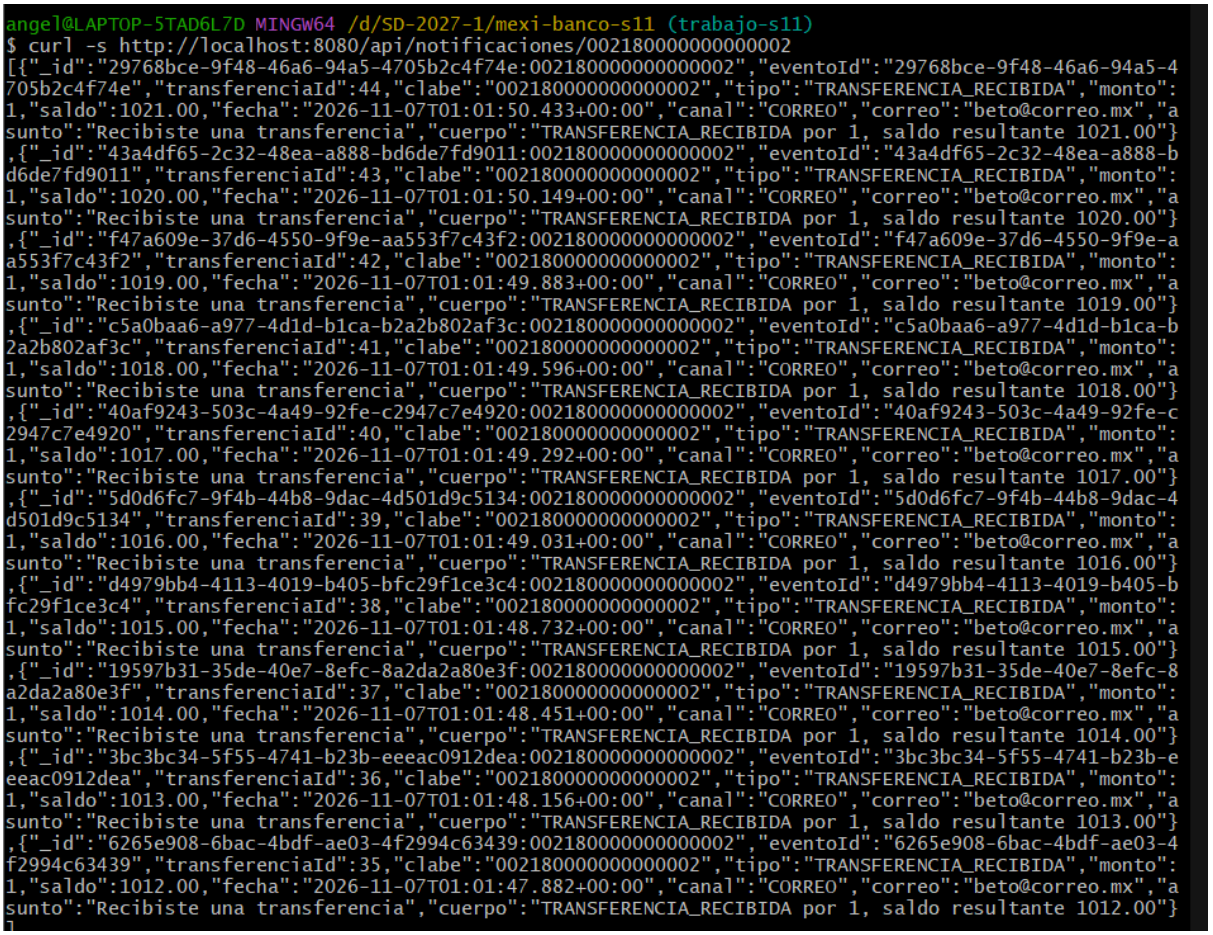


Figura 8. 10 avisos (todos con el canal CORREO), ordenados del más nuevo al más viejo.

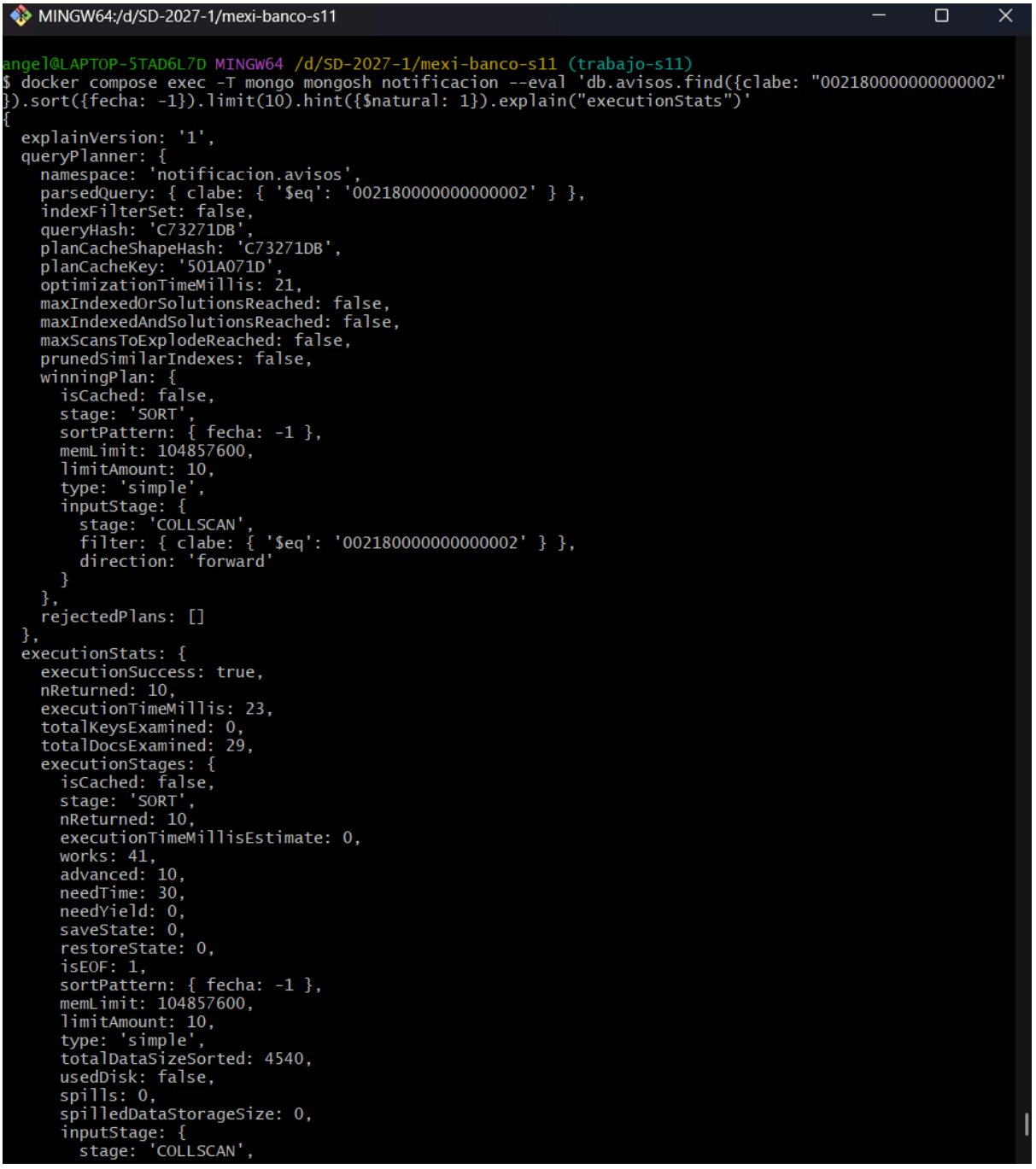


Figura 9. Respuesta de explain

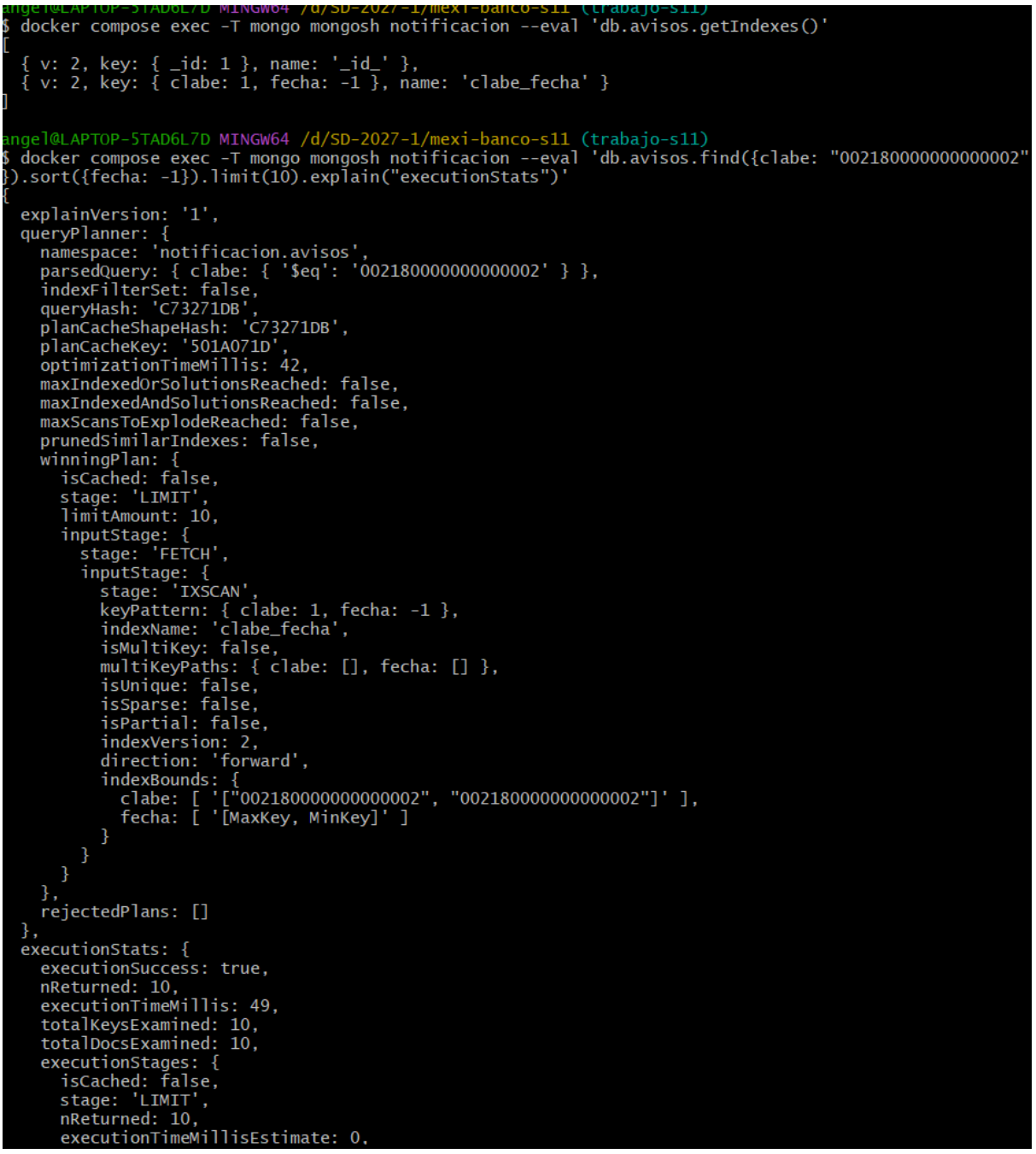


Figura 10. Respuesta de explain

5. El duplicado y el dato viejo

Se validaron dos escenarios de error crítico. Primero, al inyectar un evento de transferencia duplicado desde RabbitMQ, MongoDB lo rechazó automáticamente; gracias a que el _id del documento se construye concatenando el eventoId y la clabe, se garantizó la idempotencia sin lógica extra en el código. Segundo, se simuló un desajuste manual modificando el saldo de Ana a $5 directo en Postgres mientras Redis conservaba una copia de $200. Al intentar transferir $100, el sistema rechazó la operación con un error 422, demostrando la regla de oro: la caché solo sirve para mostrar información en pantalla, pero las decisiones de negocio (autorizar cargos) siempre leen la verdad desde la base transaccional.



Figura 11. Conteo dio 2 y las dos líneas del log diciendo que ya estaba.

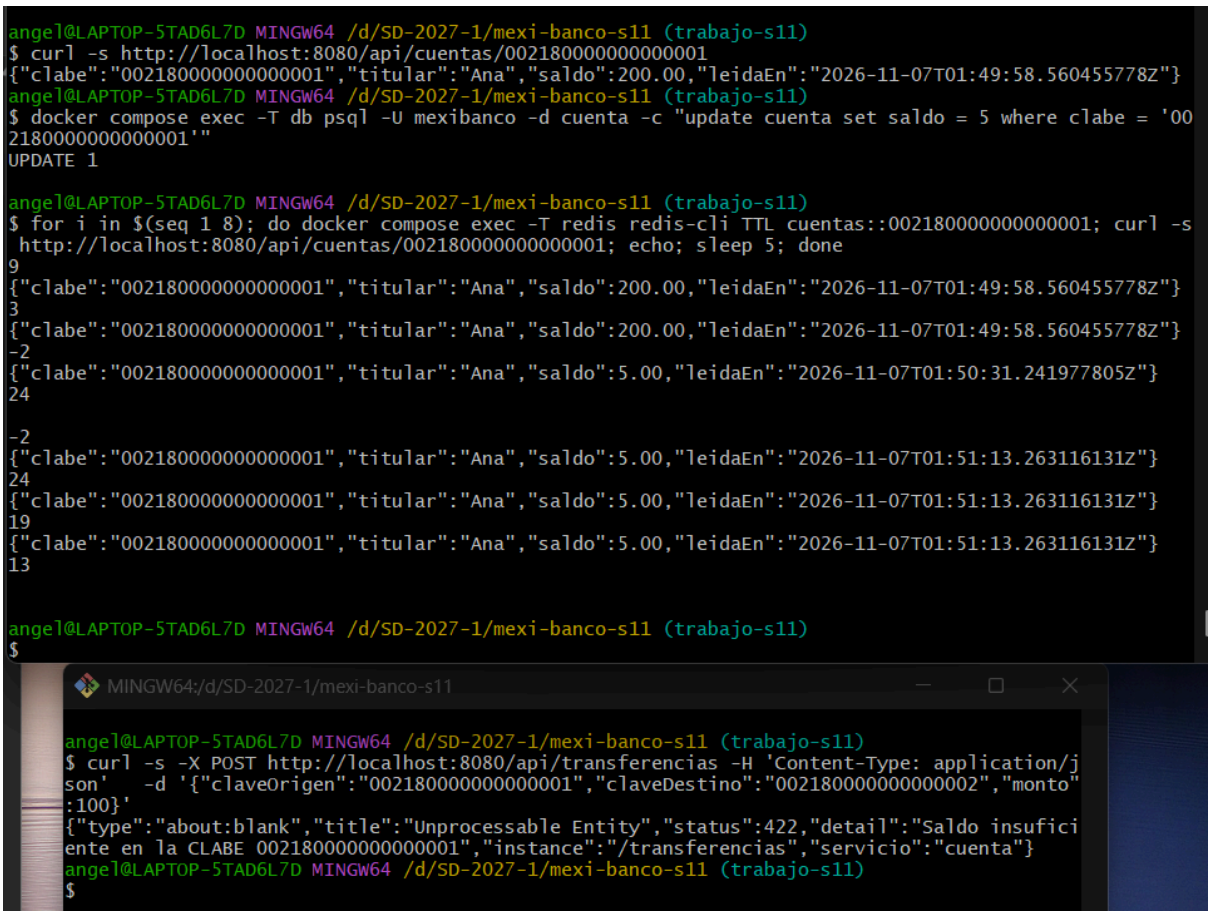


Figura 12. La terminal de arriba muestra cómo la caché mentía jurando que Ana tenía `200.00`, y la terminal de abajo demuestra que el motor del banco no se dejó engañar y rechazó la transferencia con un 422 porque en la base real solo tenía `5.00`.

Preguntas de Cierre

1. ¿Por qué MongoDB para los avisos y no para el saldo?

Porque los avisos tienen formas distintas según el canal (teléfono para SMS, correo, token para Push). MongoDB permite guardar documentos con estructuras flexibles sin tener que alterar tablas. El saldo, en cambio, maneja dinero y necesita de las garantías transaccionales estrictas (ACID) y bloqueos que ofrece PostgreSQL.

2. ¿Qué hit ratio mediste en A y en B, y por qué cambia? ¿Qué compraste con la caché?

A da ~99% y B da ~80%. Cambia porque cada transferencia en el escenario B borra la caché, provocando un fallo (miss) en la siguiente lectura. Con la caché no compramos tanta velocidad (la latencia fue casi la misma), compramos capacidad: liberamos a Postgres de hacer todas esas lecturas para que se dedique solo a procesar los cargos y abonos.

3. ¿Qué pasaría si la autorización del cargo leyera el saldo de la caché?

Pasarían fraudes y sobregiros. Como demostramos en la Sección 7, si el cargo confiara en la caché, la app habría permitido transferir los 100 pesos de Ana aunque en la base de datos real solo tuviera 5.

4.Al aplicar un movimiento, ¿por qué se hace DEL y no SET? ¿Y por qué después del commit?

Se hace DEL (borrar) en lugar de actualizar para evitar que si dos transferencias ocurren al mismo tiempo, el saldo quede desfasado. Se borra después del commit (transactionAware()) para garantizar que la caché no se vacíe antes de tiempo, lo que causaría que otra consulta lea un saldo viejo y lo vuelva a guardar en caché antes de que la transferencia se confirme.